

Perceptrón multicapa

Primero ejecuté el notebook “Multilayer perceptron”, donde se observa una gráfica suave y donde el error tiende a disminuir de forma continua, sin embargo, al añadir más capas sucede que el modelo tiende a tardar más en reducir el error, esto se puede observar mediante los escalones presentes en la figura 3, donde se “traba” en tres puntos diferentes.

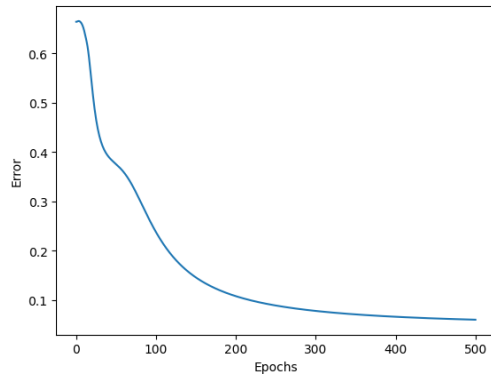


Fig 1. Multilayer perceptron original.

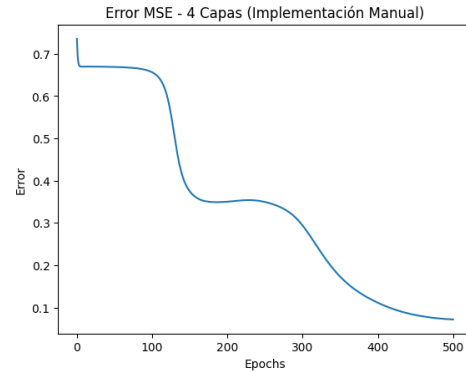


Fig 2. Multilayer perceptron con 4 capas.

En Keras ocurrió un fenómeno similar: la curva de la red profunda mostró un descenso mucho más lento y ruidoso en comparación con la curva suave y rápida de la red original. En ambos entornos, añadir profundidad empeoró la velocidad y calidad de la convergencia dentro del límite de las 500 épocas.

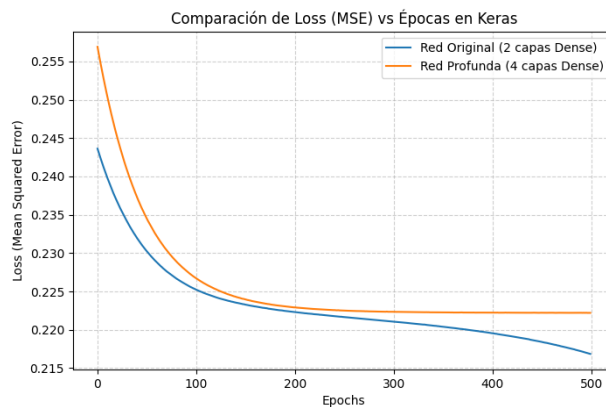


Fig 3. Red original vs red profunda.

Por otro lado, al comparar directamente las curvas de la implementación a mano con las de Keras bajo la misma topología, noté que, aunque siguen la misma tendencia teórica, visualmente no son idénticas. Esto se puede explicar por diferencias internas de implementación; por ejemplo, el código manual inicializa los pesos de forma muy básica restando 0.5 a valores aleatorios y procesa los datos de forma estrictamente secuencial. Mientras tanto, Keras, por defecto, utiliza técnicas más optimizadas como la inicialización Glorot Uniform y el entrenamiento por lotes (mini-batches) con mezcla aleatoria de los datos en cada época, lo que le ayuda a suavizar el descenso del error.

Model: "sequential"		
Layer (type)	Output Shape	Param #
layer1 (Dense)	(None, 3)	15
layer2 (Dense)	(None, 3)	12
Total params: 27 (108.00 B)		
Trainable params: 27 (108.00 B)		
Non-trainable params: 0 (0.00 B)		

Fig 4. Resumen de modelo con dos capas.

Model: "sequential_1"		
Layer (type)	Output Shape	Param #
layer1 (Dense)	(None, 3)	15
layer2 (Dense)	(None, 3)	12
layer3 (Dense)	(None, 3)	12
layer4 (Dense)	(None, 3)	12
Total params: 51 (204.00 B)		
Trainable params: 51 (204.00 B)		
Non-trainable params: 0 (0.00 B)		

Fig 5. Resumen de modelo con 4 capas.

Finalmente, que una red más profunda no aprenda mejor en este ejercicio tiene total sentido desde el punto de vista matemático. Al apilar cuatro capas utilizando la función de activación sigmoide junto con la pérdida MSE, provocamos el fenómeno del desvanecimiento del gradiente (vanishing gradient). Debido a que la derivada de la sigmoide es muy pequeña, al multiplicarse en cadena hacia atrás durante la retropropagación, las señales de corrección que llegan a las primeras capas se vuelven minúsculas, causando precisamente los estancamientos o "escalones" que vi en las gráficas. Además, dado que el conjunto de datos de Iris es un problema muy pequeño y casi linealmente separable, una red superficial tiene la capacidad suficiente para resolverlo; agregar más profundidad aquí no le da más "inteligencia" al modelo, sino que solo entorpece su proceso de aprendizaje.